Enterprise MCP Governance & Agent Control Plane

Product idea, architecture, positioning, prototype plan and preliminary prior-art findings

Working thesis: Build an enterprise MCP Gateway + Governance Control Plane that sits between AI clients/agents and MCP servers, enforcing centrally managed policy-as-code for tool access, parameters, approvals and audit.

1. The Original Idea

Create MCP servers and manage them through a GUI, with complete organizational control over which tools are exposed to LLMs. The discussion evolved from a GUI MCP builder into a broader enterprise **Agent Access Control / MCP Governance Platform**.

- Centralize MCP server and tool management.
- Enable/disable tools for specific users, agents, roles and environments.
- Use JSON/YAML policy-as-code as the underlying policy representation.
- Provide a GUI as a visual policy editor rather than making the GUI the core product.
- Enforce policies at runtime through a central MCP Gateway.
- Provide approvals, auditing, rate limits and eventually risk-based controls.

2. Product Positioning

One-line pitch: “The policy engine for AI agents.”

Enterprise pitch: Organizations increasingly give AI agents access to Jira, GitHub, email, Kubernetes, AWS, databases and internal APIs. The platform gives every agent an identity and centrally enforced policy defining exactly what it can access, what actions it can perform, which parameters are permitted, and which actions require human approval.

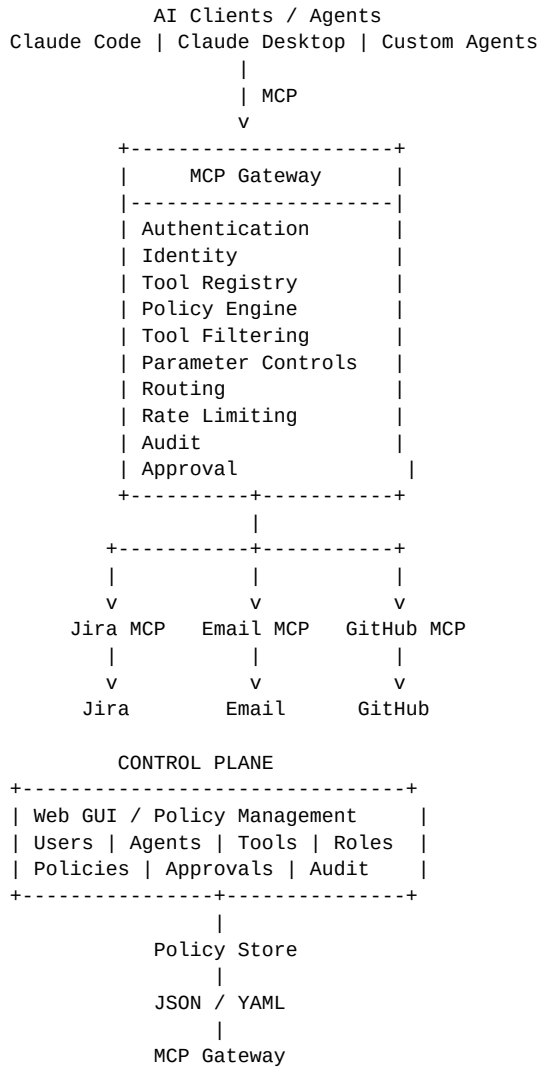
Positioning: Do not position the product merely as a “GUI for MCP servers.” Position it as an **Enterprise MCP Control Plane / Agent Access Control Platform**. MCP is the first major integration and not necessarily the entire product boundary.

3. Comparison With LiteLLM / LLM Gateways

LLM Gateway	MCP Governance Gateway
Controls model access	Controls agent/tool access
GPT / Claude / Gemini routing	Jira / Email / GitHub / Kubernetes / APIs
Model authentication & spend	Tool authorization & capability control
Model routing / fallbacks	MCP routing / tool filtering
Primary question: Which model can I use?	Primary question: What can my agent do?

The product can integrate with LLM gateways such as LiteLLM rather than necessarily competing with them. A larger enterprise architecture could have a model gateway for model access and this platform for agent/tool access.

4. Core Architecture



5. Claude Code Example

Without the platform, Claude Code connects directly to multiple MCP servers:

```
Claude Code
   |----> Jira MCP
   |
   +----> Email MCP
```

With the platform, Claude Code connects to one enterprise MCP endpoint. The gateway aggregates and proxies the downstream MCP servers.

```
Claude Code
      |
      v
Your MCP Gateway
      |
      +----> Jira MCP
      |
      +----> Email MCP
```

6. Tool-Level Governance

The gateway should filter unauthorized tools during discovery and enforce authorization again when a tool is actually invoked. This creates defense in depth.

```
Jira
  search_issues       ALLOW
  get_issue           ALLOW
  create_issue        ALLOW
  update_issue        APPROVAL
  delete_issue        DENY

Email
  search_email        ALLOW
  read_email          ALLOW
  send_email          APPROVAL
  delete_email        DENY
```

7. Policy-as-Code

```
agent:
  name: production-support

servers:
  jira:
    tools:
      search_issues:
        action: allow
      get_issue:
        action: allow
      create_issue:
        action: allow
      delete_issue:
        action: deny

  email:
    tools:
      search_emails:
        action: allow
      read_email:
        action: allow
      send_email:
        action: approval
      delete_email:
        action: deny
```

The GUI should generate/edit this policy rather than making permissions exist only inside the UI database. This enables GitOps, versioning, review and deployment workflows.

8. Advanced Policy Model

The stronger product is not just tool → allow/deny. The policy engine can evaluate:

- User identity
- Agent identity
- Role
- Tool
- Tool parameters
- Target resource
- Environment
- Session/context
- Risk level
- Approval state

```
User + Agent + Tool + Parameters + Resource + Environment + Context
                                |
                                v
                           Policy Engine
                                |
                      +---------+---------+
                      |         |         |
                    ALLOW     DENY    APPROVAL
```

9. Parameter-Level Authorization

A particularly useful capability is authorization based on tool arguments, not only the tool name.

```
email.send(recipient="employee@company.com")
    -> ALLOW

email.send(recipient="external@gmail.com")
    -> APPROVAL

email.send(to=10,000 external addresses)
    -> BLOCK

kubernetes.delete_pod(namespace="development")
    -> ALLOW

kubernetes.delete_pod(namespace="production")
    -> BLOCK
```

10. Human-in-the-Loop

High-risk actions can pause at the gateway and require explicit approval before the downstream MCP tool is called.

```
Claude Code
   |
   v
email.send(...)
   |
   v
MCP Gateway
   |
   v
Policy: APPROVAL_REQUIRED
   |
   v
Human Approval
   |
   +---- Reject
   |
   +---- Approve
           |
           v
       Email MCP
```

11. Audit and Security

- Record user, agent, MCP server, tool, parameters/metadata, decision, policy, reason and timestamp.
- Provide a complete view of what AI agents attempted and what they were permitted to do.
- Support security investigations and compliance reporting.
- Eventually integrate with SIEM and enterprise security systems.
- Add risk scoring and anomaly detection later.

12. Control Plane vs Data Plane

Control plane: Web UI, users, agents, MCP servers, tools, policies, roles, approvals, audit and policy storage.

Data plane: The high-performance MCP Gateway that authenticates, evaluates policy, filters tools, routes requests, enforces approvals and records decisions.

This separation is important for an enterprise deployment because the gateway should be able to run headlessly in Kubernetes and be managed through APIs/GitOps.

13. Prototype Plan

The agreed strategy is **prototype first, then reconsider patent protection**. Do not spend months building the full platform before validating the core mechanism.

- Build a thin MCP Gateway.
- Connect Claude Code to the gateway.
- Connect Jira MCP and Email MCP behind the gateway.
- Implement MCP proxying and tool discovery.
- Implement YAML-based allow/deny/approval policies.
- Filter unauthorized tools from tool discovery.
- Enforce authorization again on tool calls.
- Add audit logging.
- Add a minimal GUI after the gateway works.
- Progressively test user, agent, parameter, environment and approval policies.

Suggested initial stack: Python + FastAPI, official MCP Python SDK, YAML policy engine, PostgreSQL, Angular, JWT initially, and Docker Compose. Keep the first prototype deliberately small.

14. Prototype Success Criteria

```
Claude Code -> Your MCP Gateway -> Jira MCP + Email MCP

Test 1:
jira.search_issues
  -> ALLOW

Test 2:
jira.delete_issue
  -> DENY

Test 3:
email.send
  -> APPROVAL
  -> Human approves
  -> Email MCP executes

Test 4:
Unauthorized tools are not exposed through tools/list

Test 5:
Every decision is visible in the audit log
```

15. Patent / Prior-Art Findings

A preliminary web search found substantial existing prior art. The broad idea should **not** currently be treated as patentable without deeper analysis.

- Paradigm Networks patent application US20260058997A1 is extremely close to the proposed MCP gateway/policy architecture, including proxying between agents and MCP servers, policy-based tool control, tool filtering, runtime authorization, RBAC, UI-based policies and policy validation.
- Microsoft MCP Gateway provides centralized MCP management, reverse proxying and authorization capabilities.
- Kuadrant documents tool-level MCP authorization using policy rules.
- US Patent 12,688,261 covers authorization of tool invocation by autonomous AI agents and related context/least-privilege concepts.
- Airia patents cover AI-agent tool discovery, authorization and tool credentials.
- Recent 2026 research independently describes centralized enterprise MCP gateway architectures and agent/tool authorization.

Conclusion: do not try to patent “MCP gateway + YAML policies + allow/deny tools.” If a patent is eventually pursued, the likely target should be a genuinely novel technical mechanism discovered during prototype development.

16. Current Product Thesis

Product: Enterprise MCP Governance / Agent Access Control Platform.

Core: MCP Gateway + policy engine + control plane.

Policy: JSON/YAML, with GUI management and GitOps support.

Primary promise: One gateway, every MCP tool, complete control.

Long-term expansion: MCP → APIs → databases → Kubernetes → cloud → SaaS, turning MCP governance into broader AI-agent authorization.

17. Recommended Next Step

Build the smallest working version: **Claude Code** → **MCP Gateway** → **Jira MCP + Email MCP**, with YAML policies for allow/deny/approval and audit logging. During development, maintain an invention log for any technically novel mechanisms. Once the prototype reveals what is genuinely different, perform a deeper patent-claim analysis and consult a patent attorney before public disclosure.

Prepared from the product-design discussion in this conversation. Preliminary IP observations are not legal advice.